

尚硅谷大数据技术之 SparkStreaming

（作者：尚硅谷研究院）

版本：V5.0

第 1 章 SparkStreaming 概述

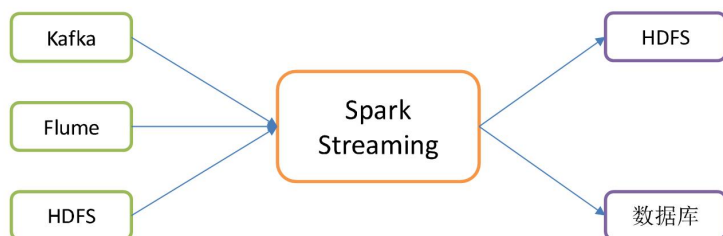
1.1 Spark Streaming 是什么

官方文档: <https://spark.apache.org/docs/latest/streaming-programming-guide.html>

Spark Streaming 是什么

Spark Streaming用于流式数据的处理。

- Spark Streaming支持的数据输入源很多，例如：Kafka、Flume、HDFS等。
- 数据输入后可以用Spark的高度抽象原语如：map、reduce、join、window等进行运算。
- 而结果也能保存在很多地方，如HDFS、数据库等。



让天下没有难学的技术

1.2 Spark Streaming 架构原理

1.2.1 什么是 DStream

Dstream是什么

SparkCore => RDD

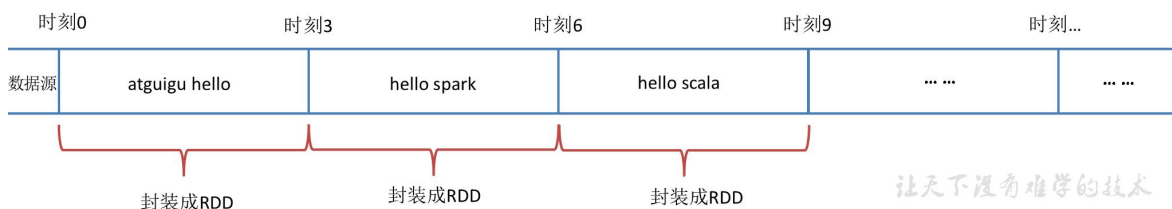
SparkSQL => DataFrame、DataSet

Spark Streaming使用离散化流（Discretized Stream）作为抽象表示，叫作DStream。

DStream是随时间推移而收到的数据的序列。

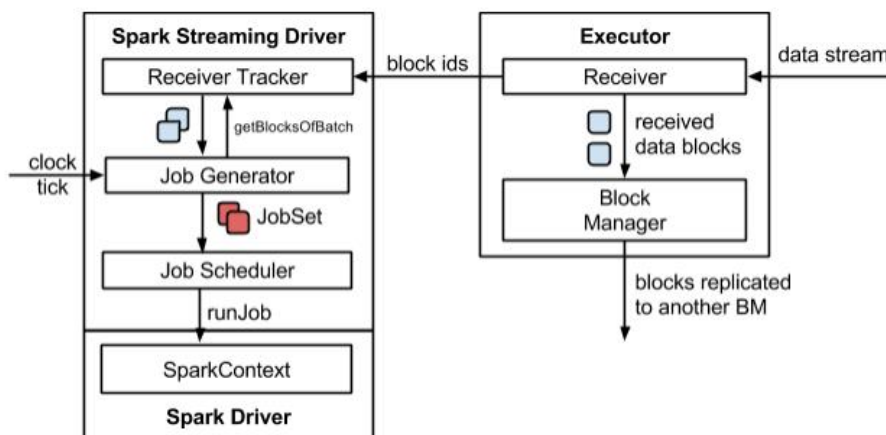
在DStream内部，每个时间区间收到的数据都作为RDD存在，而DStream是由这些RDD所组成的序列（因此得名“离散化”）。

所以简单来讲，DStream就是对RDD在实时数据处理场景的一种封装。

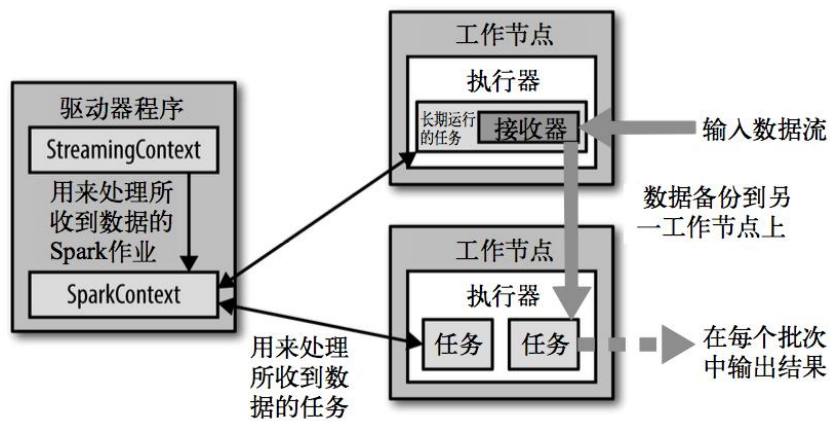


1.2.2 架构图

➤ 整体架构图



➤ SparkStreaming 架构图



1.3 Spark Streaming 特点

➤ 易用

Ease of Use

Build applications through high-level operators.

Spark Streaming brings Spark's *language-integrated API* to stream processing, letting you write streaming jobs the same way you write batch jobs. It supports Java, Scala and Python.

```
TwitterUtils.createStream(...)
  .filter(_.getText().contains("spark"))
  .countByWindow(Seconds(5))
```

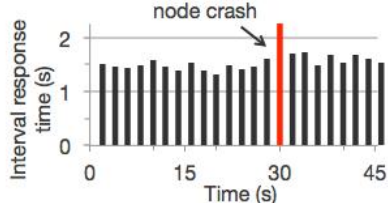
Counting tweets on a sliding window

➤ 容错

Fault Tolerance

Stateful exactly-once semantics out of the box.

Spark Streaming recovers both lost work and operator state (e.g. sliding windows) out of the box, without any extra code on your part.



The graph plots 'Interval response time (s)' on the y-axis (0 to 2) against 'Time (s)' on the x-axis (0 to 45). A vertical red line at 30 seconds marks a 'node crash'. Before the crash, the response time is stable around 1.5s. After the crash, the response time drops to 0 for a brief period before recovering to the previous level.

➤ 易整合到 Spark 体系

Spark Integration

Combine streaming with batch and interactive queries.

By running on Spark, Spark Streaming lets you reuse the same code for batch processing, join streams against historical data, or run ad-hoc queries on stream state. Build powerful interactive applications, not just analytics.

```
stream.join(historicCounts).filter {
  case (word, (curCount, oldCount)) =>
    curCount > oldCount
}
```

Find words with higher frequency than historic data

第 2 章 HelloWorld

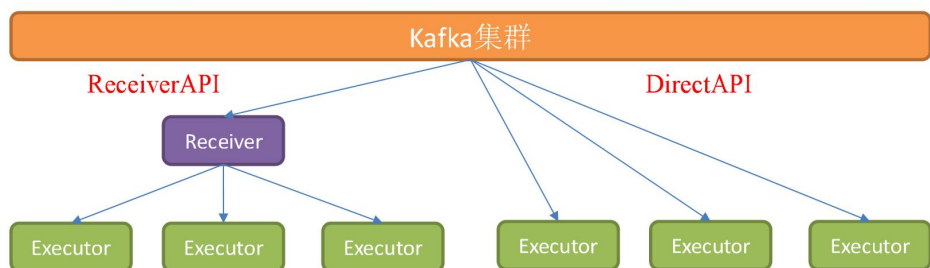
2.1 版本选型



版本选型



- **ReceiverAPI**: 需要一个专门的**Executor**去接收数据，然后发送给其他的**Executor**做计算。
存在的问题：接收数据的**Executor**和计算的**Executor**速度会有所不同，特别在接收数据的**Executor**速度大于计算的**Executor**速度，会导致计算数据的节点内存溢出。早期版本中提供此方式，当前版本不适用。
- **DirectAPI**: 是由计算的**Executor**来主动消费**Kafka**的数据，速度由自身控制。



让天下没有难学的技术

注意：目前 spark3.0.0 以上版本只有 Direct 模式。

2.2 HelloWorld 实现

- 需求：通过 **SparkStreaming** 读取 **kafka** 某个主题的数据并打印输出到控制台。

官方文档: <https://spark.apache.org/docs/latest/streaming-kafka-0-10-integration.html>

1) 添加依赖

```
<dependencies>
  <dependency>
    <groupId>org.apache.spark</groupId>
    <artifactId>spark-streaming_2.12</artifactId>
    <version>3.3.1</version>
  </dependency>

  <dependency>
    <groupId>org.apache.spark</groupId>
    <artifactId>spark-core_2.12</artifactId>
    <version>3.3.1</version>
  </dependency>

  <dependency>
    <groupId>org.apache.spark</groupId>
    <artifactId>spark-streaming-kafka-0-10_2.12</artifactId>
    <version>3.3.1</version>
  </dependency>
</dependencies>
```

2) 编写代码

```
package com.atguigu;

import org.apache.kafka.clients.consumer.ConsumerConfig;
import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.spark.api.java.function.Function;
import org.apache.spark.streaming.Duration;
import org.apache.spark.streaming.api.java.JavaInputDStream;
import org.apache.spark.streaming.api.java.JavaStreamingContext;
import org.apache.spark.streaming.kafka010.ConsumerStrategies;
import org.apache.spark.streaming.kafka010.KafkaUtils;
import org.apache.spark.streaming.kafka010.LocationStrategies;

import java.util.ArrayList;
import java.util.HashMap;

public class Test01_HelloWorld {
    public static void main(String[] args) throws InterruptedException {
        // 创建流环境
        JavaStreamingContext javaStreamingContext = new JavaStreamingContext("local[*]",
            "HelloWorld", Duration.apply(3000));

        // 创建配置参数
        HashMap<String, Object> map = new HashMap<>();

        map.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "hadoop102:9092,hadoop103:9092,hadoop104:9092");
        map.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
            "org.apache.kafka.common.serialization.StringDeserializer");
        map.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
            "org.apache.kafka.common.serialization.StringDeserializer");
        map.put(ConsumerConfig.GROUP_ID_CONFIG, "atguigu");
        map.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "latest");

        // 需要消费的主题
        ArrayList<String> strings = new ArrayList<>();
        strings.add("topic_db");

        JavaInputDStream<ConsumerRecord<String, String>> directStream =
            KafkaUtils.createDirectStream(javaStreamingContext,
                LocationStrategies.PreferBrokers(),
                ConsumerStrategies.<String, String>Subscribe(strings, map));

        directStream.map(new Function<ConsumerRecord<String, String>, String>() {
            @Override
            public String call(ConsumerRecord<String, String> v1) throws Exception {
                return v1.value();
            }
        }).print(100);

        // 执行流的任务
        javaStreamingContext.start();
        javaStreamingContext.awaitTermination();
    }
}
```

}

3) 更改日志打印级别

如果不希望运行时打印大量日志，可以在 resources 文件夹中添加 **log4j2.properties** 文件，并添加日志配置信息

```
# Set everything to be logged to the console
rootLogger.level = ERROR
rootLogger.appenderRef.stdout.ref = console

# In the pattern layout configuration below, we specify an explicit '%ex' conversion
# pattern for logging Throwables. If this was omitted, then (by default) Log4J would
# implicitly add an '%xEx' conversion pattern which logs stacktraces with additional
# class packaging information. That extra information can sometimes add a substantial
# performance overhead, so we disable it in our default logging config.
# For more information, see SPARK-39361.
appender.console.type = Console
appender.console.name = console
appender.console.target = SYSTEM_ERR
appender.console.layout.type = PatternLayout
appender.console.layout.pattern = %d{yy/MM/dd HH:mm:ss} %p %c{1}: %m%n%ex

# Set the default spark-shell/spark-sql log level to WARN. When running the
# spark-shell/spark-sql, the log level for these classes is used to overwrite
# the root logger's log level, so that the user can have different defaults
# for the shell and regular Spark apps.
logger.repl.name = org.apache.spark.repl.Main
logger.repl.level = warn
logger.thriftserver.name = org.apache.spark.sql.hive.thriftserver.SparkSQLCLIDriver
logger.thriftserver.level = warn

# Settings to quiet third party logs that are too verbose
logger.jetty1.name = org.sparkproject.jetty
logger.jetty1.level = warn
logger.jetty2.name = org.sparkproject.jetty.util.component.AbstractLifeCycle
logger.jetty2.level = error
logger.replExprTyper.name = org.apache.spark.repl.SparkIMain$ExprTyper
logger.replExprTyper.level = info
logger.replSparkILoopInterpreter.name = org.apache.spark.repl.SparkILoop$SparkILoopInterpreter
logger.replSparkILoopInterpreter.level = info
logger.parquet1.name = org.apache.parquet
logger.parquet1.level = error
logger.parquet2.name = parquet
logger.parquet2.level = error

# SPARK-9183: Settings to avoid annoying messages when looking up nonexistent UDFs in SparkSQL
with Hive support
logger.RetryingHMSHandler.name = org.apache.hadoop.hive.metastore.RetryingHMSHandler
logger.RetryingHMSHandler.level = fatal
logger.FunctionRegistry.name = org.apache.hadoop.hive.ql.exec.FunctionRegistry
logger.FunctionRegistry.level = error

# For deploying Spark ThriftServer
# SPARK-34128: Suppress undesirable TTransportException warnings involved in THRIFT-4805
appender.console.filter.1.type = RegexFilter
```

```
appender.console.filter.1.regex = .*Thrift error occurred during processing of message.*
appender.console.filter.1.onMatch = deny appender.console.filter.1.onMismatch = neutral
```

4) 启动生产者生产数据

```
[atguigu@hadoop102 ~]$ kafka-console-producer.sh --broker-list hadoop102:9092 --topic topicA
hello spark
```

5) 在 Idea 控制台输出如下内容

```
-----
Time: 1602731772000 ms
-----
```

```
hello spark
```

2.3 HelloWorld 解析

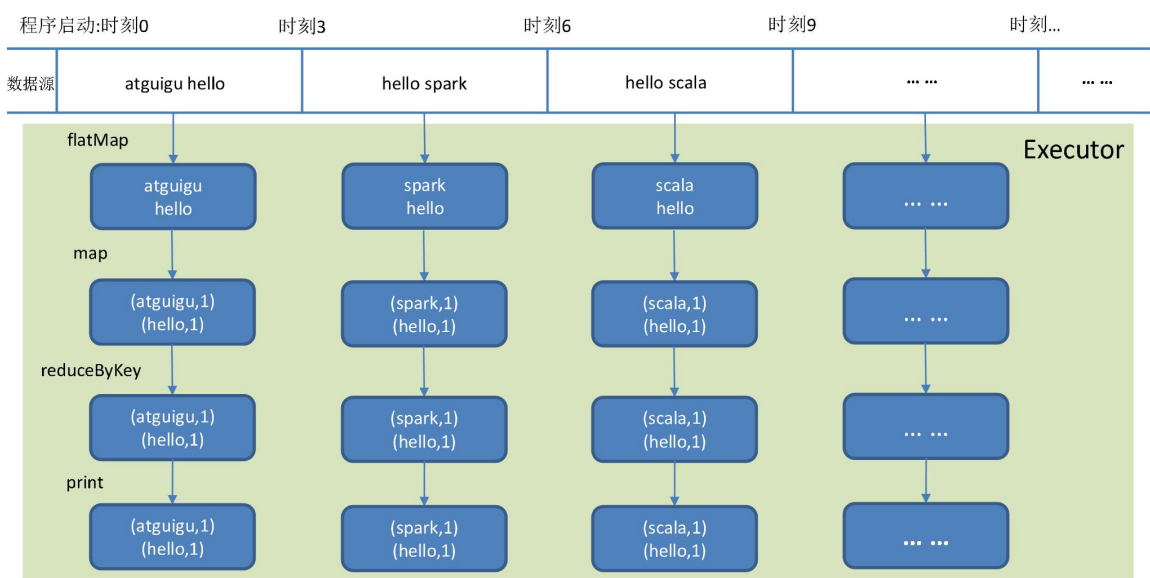
DStream 是 Spark Streaming 的基础抽象，代表持续性的数据流和经过各种 Spark 算子操作后的结果数据流。

在内部实现上，每一批次的数据封装成一个 RDD，一系列连续的 RDD 组成了 DStream。对这些 RDD 的转换是由 Spark 引擎来计算。

说明：DStream 中批次与批次之间计算相互独立。如果批次设置时间小于计算时间会出现计算任务叠加情况，需要多分配资源。通常情况，批次设置时间要大于计算时间。



WordCount 数据流解析



第 3 章 DStream 转换

DStream 上的操作与 RDD 的类似，分为转换和输出两种，此外转换操作中还有一些比较特殊的原语，如：transform() 以及各种 Window 相关的原语。

3.1 无状态转化操作

无状态转化操作：就是把 RDD 转化操作应用到 DStream 每个批次上，**每个批次相互独立，自己算自己的。**

3.1.1 常规无状态转化操作

DStream 的部分无状态转化操作列在了下表中，都是 DStream 自己的 API。

注意，只有 `JavaPairDStream<Key, Value>` 才能使用 `xxxByKey()` 类型的转换算子。

函数名称	目的	函数类型
<code>map()</code>	对 DStream 中的每个元素应用给定函数，返回由各元素输出的元素组成的 DStream。	<code>Function<in, out></code>
<code>flatMap()</code>	对 DStream 中的每个元素应用给定函数，返回由各元素输出的迭代器组成的 DStream。	<code>FlatMapFunction<in, out></code>
<code>filter()</code>	返回由给定 DStream 中通过筛选的元素组成的 DStream	<code>Function<in, Boolean></code>
<code>mapToPair()</code>	改变 DStream 的分区数	<code>PairFunction<in, key, value></code>
<code>reduceByKey()</code>	将每个批次中键相同的记录规约。	<code>Function2<in, in, in></code>
<code>groupByKey()</code>	将每个批次中的记录根据键分组。	<code>ds.groupByKey()</code>

需要记住的是，尽管这些函数看起来像作用在整个流上一样，但事实上每个 DStream 在内部是由许多 RDD **批次** 组成，且**无状态转化操作**是分别应用到**每个 RDD(一个批次的**数据**)**上的。

3.2 窗口操作

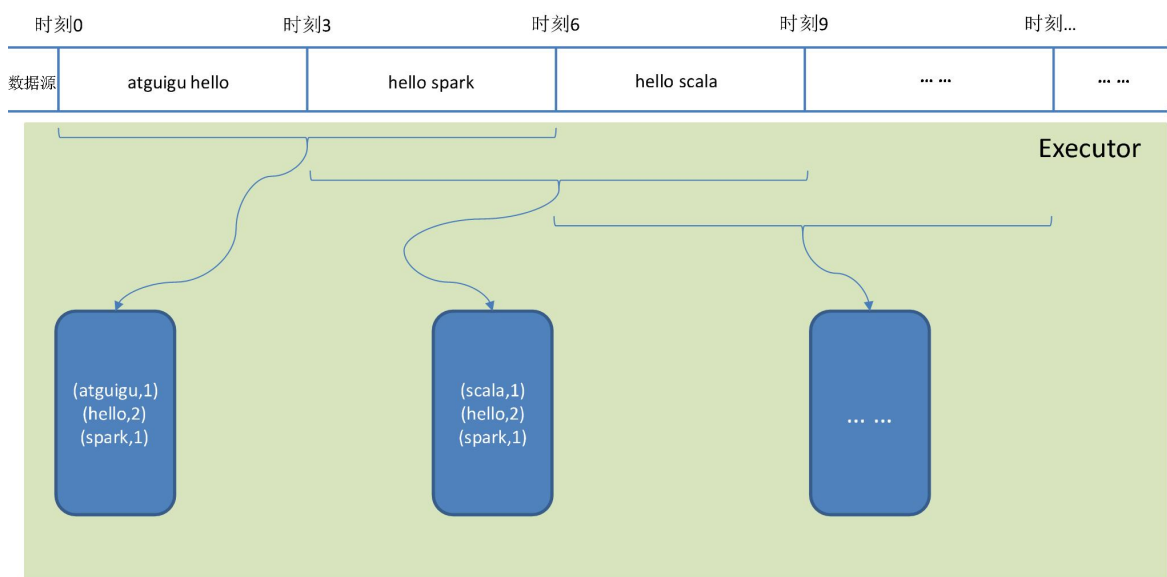
3.2.1 WindowOperations

Window Operations 可以设置窗口的大小和滑动窗口的间隔来动态的获取当前 Streaming 的允许状态。所有基于窗口的操作都需要两个参数，分别为窗口时长以及滑动步长。

- 窗口时长：计算内容的时间范围；
- 滑动步长：隔多久触发一次计算。

注意：这两者都必须为采集批次大小的整数倍。

如下图所示 WordCount 案例：窗口大小为批次的 2 倍，滑动步等于批次大小。



3.2.2 Window

1) 基本语法:

`window(windowLength, slideInterval)`: 基于对源 DStream 窗口的批次进行计算返回一个新的 DStream。

2) 需求:

统计 WordCount: 3 秒一个批次, 窗口 12 秒, 滑步 6 秒。

3) 代码编写:

```
package com.atguigu;

import org.apache.kafka.clients.consumer.ConsumerConfig;
import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.spark.api.java.function.FlatMapFunction;
import org.apache.spark.api.java.function.Function2;
import org.apache.spark.api.java.function.PairFunction;
import org.apache.spark.streaming.Duration;
import org.apache.spark.streaming.api.java.JavaDStream;
import org.apache.spark.streaming.api.java.JavaInputDStream;
import org.apache.spark.streaming.api.java.JavaPairDStream;
import org.apache.spark.streaming.api.java.JavaStreamingContext;
import org.apache.spark.streaming.kafka010.ConsumerStrategies;
import org.apache.spark.streaming.kafka010.KafkaUtils;
import org.apache.spark.streaming.kafka010.LocationStrategies;
import scala.Tuple2;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.Iterator;
```

```
public class Test02 Window {
    public static void main(String[] args) throws InterruptedException {
        // 创建流环境
        JavaStreamingContext javaStreamingContext = new JavaStreamingContext("local[*]",
"window", Duration.apply(3000));
        // 创建配置参数
        HashMap<String, Object> map = new HashMap<>();

map.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG,"hadoop102:9092,hadoop103:9092,had
oop104:9092");
        map.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
"org.apache.kafka.common.serialization.StringDeserializer");
        map.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
"org.apache.kafka.common.serialization.StringDeserializer");
        map.put(ConsumerConfig.GROUP_ID_CONFIG,"atguigu");
        map.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG,"latest");

        // 需要消费的主题
        ArrayList<String> strings = new ArrayList<>();
        strings.add("topicA");

        JavaInputDStream<ConsumerRecord<String, String>> directStream =
KafkaUtils.createDirectStream(javaStreamingContext, LocationStrategies.PreferBrokers(),
ConsumerStrategies.<String, String>Subscribe(strings,map));

        JavaDStream<String> stringJavaDStream = directStream.flatMap(new
FlatMapFunction<ConsumerRecord<String, String>, String>() {
            @Override
            public Iterator<String> call(ConsumerRecord<String, String>
stringStringConsumerRecord) throws Exception {
                String[] split = stringStringConsumerRecord.value().split(" ");
                return Arrays.asList(split).iterator();
            }
        });

        JavaPairDStream<String, Integer> javaPairDStream = stringJavaDStream.mapToPair(new
PairFunction<String, String, Integer>() {
            @Override
            public Tuple2<String, Integer> call(String s) throws Exception {
                return new Tuple2<>(s, 1);
            }
        });

        JavaPairDStream<String, Integer> window =
javaPairDStream.window(Duration.apply(12000), Duration.apply(6000));
        window.reduceByKey(new Function2<Integer, Integer, Integer>() {
            @Override
            public Integer call(Integer v1, Integer v2) throws Exception {
                return v1+v2;
            }
        }).print();

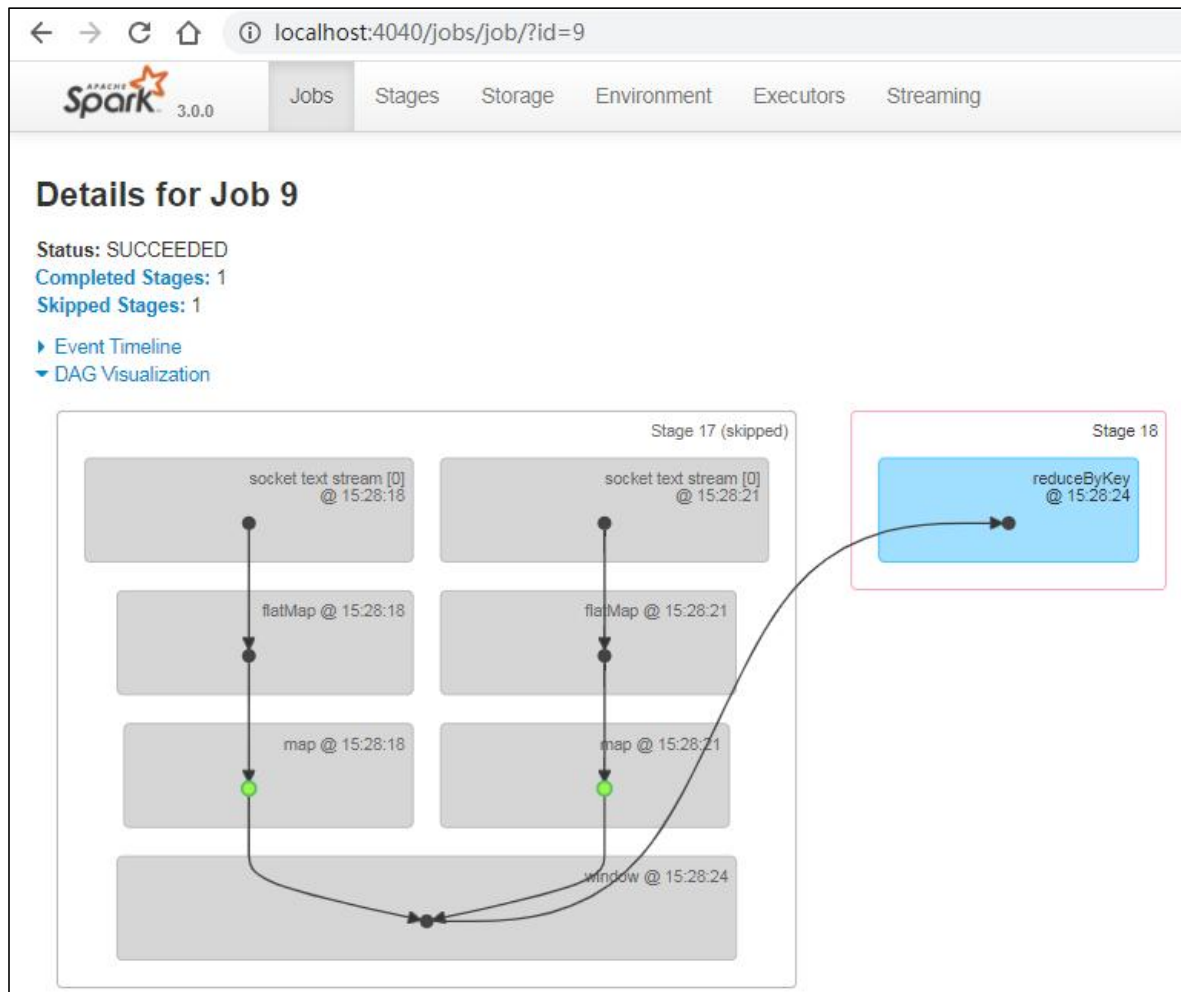
        // 执行流的任务
        javaStreamingContext.start();
    }
}
```

```
javaStreamingContext.awaitTermination();
}
}
```

4) 测试

```
[atguigu@hadoop102 ~]$ kafka-console-producer.sh --broker-list hadoop102:9092 --topic topicA
hello spark
```

5) 如果有多批数据进入窗口，最终也会通过 **window** 操作变成统一的 **RDD** 处理。



3.2.3 reduceByKeyAndWindow

1) 基本语法:

- **reduceByKeyAndWindow**(func, windowLength, slideInterval, [numTasks]): 当在一个(K,V)对的 DStream 上调用此函数，会返回一个新(K,V)对的 DStream，此处通过对滑动窗口中批次数据使用 reduce 函数来整合每个 key 的 value 值。

2) 需求:

统计 WordCount: 3 秒一个批次，窗口 12 秒，滑步 6 秒。

3) 代码编写:

```
package com.atguigu;

import org.apache.kafka.clients.consumer.ConsumerConfig;
import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.spark.api.java.function.FlatMapFunction;
import org.apache.spark.api.java.function.Function2;
import org.apache.spark.api.java.function.PairFunction;
import org.apache.spark.streaming.Duration;
import org.apache.spark.streaming.api.java.JavaDStream;
import org.apache.spark.streaming.api.java.JavaInputDStream;
import org.apache.spark.streaming.api.java.JavaPairDStream;
import org.apache.spark.streaming.api.java.JavaStreamingContext;
import org.apache.spark.streaming.kafka010.ConsumerStrategies;
import org.apache.spark.streaming.kafka010.KafkaUtils;
import org.apache.spark.streaming.kafka010.LocationStrategies;
import scala.Tuple2;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.Iterator;

public class Test03_ReduceByKeyAndWindow {
    public static void main(String[] args) throws InterruptedException {
        // 创建流环境
        JavaStreamingContext javaStreamingContext = new JavaStreamingContext("local[*]",
            "window", Duration.apply(3000L));
        // 创建配置参数
        HashMap<String, Object> map = new HashMap<>();

        map.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "hadoop102:9092,hadoop103:9092,hadoop104:9092");
        map.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
            "org.apache.kafka.common.serialization.StringDeserializer");
        map.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
            "org.apache.kafka.common.serialization.StringDeserializer");
        map.put(ConsumerConfig.GROUP_ID_CONFIG, "atguigu");
        map.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "latest");

        // 需要消费的主题
        ArrayList<String> strings = new ArrayList<>();
        strings.add("topicA");

        JavaInputDStream<ConsumerRecord<String, String>> directStream =
            KafkaUtils.createDirectStream(javaStreamingContext,
                LocationStrategies.PreferBrokers(),
                ConsumerStrategies.<String, String>Subscribe(strings, map));

        JavaDStream<String> stringJavaDStream = directStream.flatMap(new
            FlatMapFunction<ConsumerRecord<String, String>, String>() {
                @Override
                public Iterator<String> call(ConsumerRecord<String, String>
                    stringStringConsumerRecord) throws Exception {
                    String[] split = stringStringConsumerRecord.value().split(" ");
                    return Arrays.asList(split).iterator();
                }
            });
    }
}
```

```
    }
    });

    JavaPairDStream<String, Integer> javaPairDStream = stringJavaDStream.mapToPair(new
PairFunction<String, String, Integer>() {
        @Override
        public Tuple2<String, Integer> call(String s) throws Exception {
            return new Tuple2<>(s, 1);
        }
    });

    javaPairDStream.reduceByKeyAndWindow(new Function2<Integer, Integer, Integer>() {
        @Override
        public Integer call(Integer v1, Integer v2) throws Exception {
            return v1 + v2;
        }
    }, Duration.apply(12000L), Duration.apply(6000L)).print();

    // 执行流的任务
    javaStreamingContext.start();
    javaStreamingContext.awaitTermination();
}
}
```

2) 测试

```
[atguigu@hadoop102 ~]$ kafka-console-producer.sh --broker-list hadoop102:9092 --topic topicA
hello spark
```

第 4 章 DStream 输出

DStream 通常将数据输出到，**外部数据库或屏幕上**。

DStream 与 RDD 中的**惰性**求值类似，如果一个 DStream 及其派生出的 DStream 都没有被执行输出操作，那么这些 DStream 就都不会被求值。如果 StreamingContext 中没有设定输出操作，整个 Context 就都不会启动。

1) 输出操作 API 如下：

- `saveAsTextFiles(prefix, [suffix])`：以 text 文件形式存储这个 DStream 的内容。**每一批次**的存储文件名基于参数中的 prefix 和 suffix。“prefix-Time_IN_MS[suffix]”。

注意：以上操作都是每一批次写出一，会产生大量小文件，在生产环境，很少使用。

- `print()`：在运行程序的驱动节点上打印 DStream 中**每一批次数据的最开始 10 个元素**。这用于开发和调试。

- **foreachRDD(func)**: 这是最通用的输出操作, 即将函数 func 用于产生 DStream 的每一个 RDD。其中参数传入的函数 func 应该实现将每一个 RDD 中数据推送到外部系统, 如将 RDD 存入文件或者写入数据库。

在企业开发中通常采用 foreachRDD(), 它用来对 DStream 中的 RDD 进行任意计算。这和 transform() 有些类似, 都可以让我们访问任意 RDD。在 foreachRDD() 中, 可以重用我们在 Spark 中实现的所有行动操作(action 算子)。比如, 常见的用例之一是把数据写到如 MySQL 的外部数据库中。

```
package com.atguigu;

import org.apache.kafka.clients.consumer.ConsumerConfig;
import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.spark.api.java.JavaPairRDD;
import org.apache.spark.api.java.function.FlatMapFunction;
import org.apache.spark.api.java.function.Function2;
import org.apache.spark.api.java.function.PairFunction;
import org.apache.spark.api.java.function.VoidFunction;
import org.apache.spark.streaming.Duration;
import org.apache.spark.streaming.api.java.JavaDStream;
import org.apache.spark.streaming.api.java.JavaInputDStream;
import org.apache.spark.streaming.api.java.JavaPairDStream;
import org.apache.spark.streaming.api.java.JavaStreamingContext;
import org.apache.spark.streaming.kafka010.ConsumerStrategies;
import org.apache.spark.streaming.kafka010.KafkaUtils;
import org.apache.spark.streaming.kafka010.LocationStrategies;
import scala.Tuple2;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.Iterator;

/**
 * @author yhm
 * @create 2022-09-01 16:47
 */
public class Test04_Save {
    public static void main(String[] args) throws InterruptedException {
        // 创建流环境
        JavaStreamingContext javaStreamingContext = new JavaStreamingContext("local[*]",
            "window", Duration.apply(3000L));
        // 创建配置参数
        HashMap<String, Object> map = new HashMap<>();

        map.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "hadoop102:9092,hadoop103:9092,hadoop104:9092");
        map.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
            "org.apache.kafka.common.serialization.StringDeserializer");
        map.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
            "org.apache.kafka.common.serialization.StringDeserializer");
```

```
map.put(ConsumerConfig.GROUP_ID_CONFIG,"atguigu");
map.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG,"latest");

// 需要消费的主题
ArrayList<String> strings = new ArrayList<>();
strings.add("topicA");

JavaInputDStream<ConsumerRecord<String, String>> directStream =
KafkaUtils.createDirectStream(javaStreamingContext, LocationStrategies.PreferBrokers(),
ConsumerStrategies.<String, String>Subscribe(strings,map));

JavaDStream<String> stringJavaDStream = directStream.flatMap(new
FlatMapFunction<ConsumerRecord<String, String>, String>() {
    @Override
    public Iterator<String> call(ConsumerRecord<String, String>
stringStringConsumerRecord) throws Exception {
        String[] split = stringStringConsumerRecord.value().split(" ");
        return Arrays.asList(split).iterator();
    }
});

JavaPairDStream<String, Integer> javaPairDStream = stringJavaDStream.mapToPair(new
PairFunction<String, String, Integer>() {
    @Override
    public Tuple2<String, Integer> call(String s) throws Exception {
        return new Tuple2<>(s, 1);
    }
});

JavaPairDStream<String, Integer> resultDStream =
javaPairDStream.reduceByKeyAndWindow(new Function2<Integer, Integer, Integer>() {
    @Override
    public Integer call(Integer v1, Integer v2) throws Exception {
        return v1 + v2;
    }
}, Duration.apply(12000L), Duration.apply(6000L));

resultDStream.foreachRDD(new VoidFunction<JavaPairRDD<String, Integer>>() {
    @Override
    public void call(JavaPairRDD<String, Integer> stringIntegerJavaPairRDD) throws
Exception {
        // 获取 mysql 连接
        // 写入到 mysql 中
        // 关闭连接
    }
});

// 执行流的任务
javaStreamingContext.start();
javaStreamingContext.awaitTermination();
}
```

第 5 章 优雅关闭

流式任务需要 7*24 小时执行，但是有时涉及到 **升级代码** 需要 **主动停止程序**，但是分布式程序，没办法做到一个个进程去杀死，所以配置优雅的关闭就显得至关重要了。

关闭方式：使用外部文件系统来控制内部程序关闭。

1) 主程序

```
package com.atguigu;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.Path;
import org.apache.kafka.clients.consumer.ConsumerConfig;
import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.spark.api.java.function.FlatMapFunction;
import org.apache.spark.api.java.function.Function2;
import org.apache.spark.api.java.function.PairFunction;
import org.apache.spark.streaming.Duration;
import org.apache.spark.streaming.StreamingContextState;
import org.apache.spark.streaming.api.java.JavaDStream;
import org.apache.spark.streaming.api.java.JavaInputDStream;
import org.apache.spark.streaming.api.java.JavaPairDStream;
import org.apache.spark.streaming.api.java.JavaStreamingContext;
import org.apache.spark.streaming.kafka010.ConsumerStrategies;
import org.apache.spark.streaming.kafka010.KafkaUtils;
import org.apache.spark.streaming.kafka010.LocationStrategies;
import scala.Tuple2;

import java.net.URI;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.Iterator;

public class Test05_Close {
    public static void main(String[] args) throws InterruptedException {
        // 创建流环境
        JavaStreamingContext javaStreamingContext = new JavaStreamingContext("local[*]",
            "window", Duration.apply(3000L));
        // 创建配置参数
        HashMap<String, Object> map = new HashMap<>();

        map.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "hadoop102:9092,hadoop103:9092,hadoop104:9092");
        map.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
            "org.apache.kafka.common.serialization.StringDeserializer");
        map.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
            "org.apache.kafka.common.serialization.StringDeserializer");
        map.put(ConsumerConfig.GROUP_ID_CONFIG, "atguigu");
        map.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "latest");
```

```
// 需要消费的主题
ArrayList<String> strings = new ArrayList<>();
strings.add("topicA");

JavaInputDStream<ConsumerRecord<String, String>> directStream =
KafkaUtils.createDirectStream(javaStreamingContext, LocationStrategies.PreferBrokers(),
ConsumerStrategies.<String, String>Subscribe(strings,map));

JavaDStream<String> stringJavaDStream = directStream.flatMap(new
FlatMapFunction<ConsumerRecord<String, String>, String>() {
    @Override
    public Iterator<String> call(ConsumerRecord<String, String>
stringStringConsumerRecord) throws Exception {
        String[] split = stringStringConsumerRecord.value().split(" ");
        return Arrays.asList(split).iterator();
    }
});

JavaPairDStream<String, Integer> javaPairDStream = stringJavaDStream.mapToPair(new
PairFunction<String, String, Integer>() {
    @Override
    public Tuple2<String, Integer> call(String s) throws Exception {
        return new Tuple2<>(s, 1);
    }
});

javaPairDStream.reduceByKeyAndWindow(new Function2<Integer, Integer, Integer>() {
    @Override
    public Integer call(Integer v1, Integer v2) throws Exception {
        return v1 + v2;
    }
},Duration.apply(12000L),Duration.apply(6000L)).print();

// 开启监控程序
new Thread(new MonitorStop(javaStreamingContext)).start();

// 执行流的任务
javaStreamingContext.start();
javaStreamingContext.awaitTermination();
}

public static class MonitorStop implements Runnable {

    JavaStreamingContext javaStreamingContext = null;

    public MonitorStop(JavaStreamingContext javaStreamingContext) {
        this.javaStreamingContext = javaStreamingContext;
    }

    @Override
    public void run() {
        try {
            FileSystem fs = FileSystem.get(new URI("hdfs://hadoop102:8020"), new
Configuration(), "atguigu");
            while (true){
```

```
        Thread.sleep(5000);
        boolean exists = fs.exists(new Path("hdfs://hadoop102:8020/stopSpark"));
        if (exists){
            StreamingContextState state = javaStreamingContext.getState();
            // 获取当前任务是否正在运行
            if (state == StreamingContextState.ACTIVE){
                // 优雅关闭
                javaStreamingContext.stop(true, true);
                System.exit(0);
            }
        }
    }
} catch (Exception e){
    e.printStackTrace();
}
}
```

2) 测试

(1) 发送数据

```
[atguigu@hadoop102 ~]$ kafka-console-producer.sh --broker-list hadoop102:9092 --topic topicA
hello spark
```

(2) 启动 Hadoop 集群

```
[atguigu@hadoop102 hadoop-3.1.3]$ sbin/start-dfs.sh
```

```
[atguigu@hadoop102 hadoop-3.1.3]$ hadoop fs -mkdir /stopSpark
```